

ekantipur.com Scraper

Technical Documentation

Data Extraction Intern — Practical Assessment

Audio Bee

Author	Anuj Shrestha
Date	May 2026
Target	https://ekantipur.com
Stack	Python 3.12 · Playwright · uv
Run with	<code>uv run python scraper.py</code>

Abstract

This document provides a complete technical account of the web scraping solution developed for the Audio Bee Data Extraction Intern assessment. It covers the manual website inspection process used to discover HTML selectors, the AI-assisted development workflow in Cursor IDE, a line-by-line explanation of every function in `scraper.py`, the lazy-loading problem and its solution, the dynamic category extraction strategy, and the complete output schema. The solution extracts five entertainment news articles and the most recent cartoon from `ekantipur.com` without any hardcoded data values.

Contents

1	Project Overview	2
1.1	Objective	2
1.2	Technology Stack	2
1.3	Project Structure	2
2	Environment Setup	2
2.1	Installing uv	2
2.2	Creating the Project	3
2.3	Verifying the Installation	3
3	Website Inspection	3
3.1	Entertainment Section	3
3.1.1	Article Card Structure	4
3.1.2	Lazy Loading Discovery	4
3.1.3	Category Field Discovery	4
3.2	Cartoon Section	4
3.2.1	Cartoon Card Structure	5
3.2.2	Caption Parsing	5
4	AI-Assisted Development with Cursor	5
4.1	What is Cursor?	5
4.2	Development Workflow	5
4.3	Prompts Used	6
5	Code Explanation	6
5.1	Module Docstring and Imports	6
5.2	URL Constants	6
5.3	<code>_category_from_url</code> — URL Fallback	7
5.4	<code>_scroll_in_six_steps</code> — Lazy Load Handler	7
5.5	<code>_parse_cartoon_caption</code> — Caption Parser	8
5.6	<code>scrape_entertainment</code> — Main Extraction Function	8
5.7	<code>scrape_cartoon</code> — Cartoon Extraction	9
5.8	<code>main</code> — Orchestration	10
6	Error Handling Strategy	11
7	Output Format	11
7.1	Schema	11
7.2	Sample Output	12
8	Challenges and Solutions	12
9	Submission Checklist	13
10	Running the Script	13

1 Project Overview

1.1 Objective

The task requires a Python script that uses Playwright to extract structured data from two sections of ekantipur.com — a major Nepali-language news website — and save the result as a valid JSON file. Specifically:

1. **Task 1:** Navigate to the Entertainment (*manoranjani*) section and extract the top 5 news articles, each with title, image URL, category label, and author name.
2. **Task 2:** Navigate to the Cartoon section and extract the most recent cartoon with its title, image URL, and cartoonist name.

1.2 Technology Stack

Tool	Version	Purpose
Python	3.12.2	Runtime language
Playwright	1.59.0	Browser automation
uv	0.11.14	Package and project manager
Chromium	147.0.7727.15	Headless-capable browser
Cursor IDE	3.4.17	AI-assisted code editor

Table 1: Technology stack used in this project

1.3 Project Structure

```
ekantipur-scraper/  
|-- pyproject.toml      # Project config, auto-generated by uv  
|-- uv.lock             # Locked dependency versions  
|-- scraper.py          # Main scraping script  
|-- output.json         # Generated output (run the script to  
    produce)  
|-- prompts.txt         # All AI prompts used during  
    development  
|-- main.py             # Auto-generated by uv init (  
    boilerplate only)  
|-- README.md           # Auto-generated by uv init
```

2 Environment Setup

2.1 Installing uv

uv is a fast Python package and project manager written in Rust. It replaces pip, virtualenv, and pyenv in a single tool. On Windows (PowerShell):

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv  
/install.ps1 | iex"
```

After installation, add uv to the current session's PATH:

```
$env:Path = "C:\Users\<YourName>\.local\bin;$env:Path"
```

To make this permanent across all terminal sessions:

```
[Environment]::SetEnvironmentVariable(  
    "Path",  
    $env:Path + ";C:\Users\<YourName>\.local\bin",  
    "User"  
)
```

2.2 Creating the Project

```
mkdir ekantipur-scraper  
cd ekantipur-scraper  
uv init                # Creates pyproject.toml, main.py, README.  
                        md  
uv add playwright      # Installs Playwright into the virtual  
                        environment  
uv run playwright install chromium # Downloads Playwright's  
                        Chromium
```

Note on uv init

Running `uv init` automatically creates a `main.py` file containing a hello-world boilerplate and the `if __name__ == "__main__"` guard. This is expected behaviour — it is not part of the scraper and can be ignored.

2.3 Verifying the Installation

```
uv run python -c "from playwright.sync_api import  
    sync_playwright; print('OK')"  
# Expected output: OK
```

3 Website Inspection

Before writing a single line of code, the target website was manually inspected using browser DevTools (F12) with the element picker (Ctrl+Shift+C). This is a critical step — correct selectors make the scraper robust; wrong selectors make it fragile.

3.1 Entertainment Section

URL: <https://ekantipur.com/entertainment>

The page uses JavaScript hydration — the article cards are injected into the DOM after the initial HTML response. This means a scraper must *wait* for the cards to appear, not just wait for the page to load.

3.1.1 Article Card Structure

Each article card is wrapped in `div.category-inner-wrapper`. The full hierarchy is:

```
div.category-inner-wrapper      <-- one per article card
|-- div.category-description
|   |-- h2
|   |   '-- a[href]            <-- article title text +
|       URL
|   |-- div.author-name
|   |   '-- p
|   |   '-- a                  <-- author name (optional)
|   |-- p
|   '-- div.time-wrapper
'-- div.category-image
    '-- a[href]
        '-- figure
            '-- img[src]        <-- thumbnail image URL
```

3.1.2 Lazy Loading Discovery

During inspection, article images were found to have `class="loaded"` only *after* scrolling past them. This is a standard lazy-loading pattern — the browser assigns a real `src` to an `img` tag only when it enters the viewport. Without scrolling, `get_attribute("src")` returns `None` for all but the first one or two images.

Solution: Scroll the page in six incremental steps before reading any image attributes, then wait 2000ms for the browser to finish loading.

3.1.3 Category Field Discovery

No per-card category label exists in the HTML. After inspecting multiple articles, the category was confirmed to be consistent for the entire page. The section name is available in the page header:

```
header.detail-header
'-- div.container
    '-- div.category-name
        '-- p
            '-- a                <-- contains the section
                                name, e.g. "manoranjanl"
```

Selector: `div.category-name p a`

This is extracted dynamically at runtime — if the scraper is pointed at a different section (e.g., sports), it reads that section's name automatically. No string is hardcoded.

3.2 Cartoon Section

URL: <https://ekantipur.com/cartoon>

The page lists multiple cartoons going back several weeks, one per day. There is no explicit “Cartoon of the Day” label. The most recent cartoon is always the first `div.cartoon-wrapper` on the page.

3.2.1 Cartoon Card Structure

```
div.cartoon-wrapper          <-- one per cartoon
|-- div.cartoon-image
|   '-- figure
|       '-- a[href]
|           '-- img[src]    <-- cartoon image URL
|-- div.cartoon-description
|   |-- p                   <-- combined title +
|       author text
|   '-- div.date
```

3.2.2 Caption Parsing

The `<p>` inside `div.cartoon-description` contains both the title and the cartoonist name in a single string. Three formats were observed:

Raw Text	Title	Author
Gajab chha ba! - Avin	Gajab chha ba!	Avin
Gajab chha ba! -	Gajab chha ba!	null
Gajab chha ba!	Gajab chha ba!	null

Table 2: Cartoon caption parsing cases

4 AI-Assisted Development with Cursor

4.1 What is Cursor?

Cursor is a VS Code-based IDE with a built-in AI chat panel. It supports multiple modes — *Ask*, *Agent*, and *Plan* — each with different levels of autonomy. *Ask* mode generates code in the chat without touching any files; *Agent* mode can read, create, and edit files autonomously.

4.2 Development Workflow

The workflow followed in this project:

1. **Manual inspection first.** All selectors were found using browser DevTools before any AI interaction. This is the most important step — AI cannot reliably guess HTML selectors for a live website it cannot visit.
2. **One comprehensive prompt.** Instead of many back-and-forth messages (which consume free-tier credits), a single prompt was crafted containing all findings from the manual inspection, complete with selector paths, extraction logic, output schema, and all technical requirements.

3. **Review before applying.** The generated code was reviewed before pasting into `scraper.py`.
4. **Comment enhancement.** A second prompt requested improved inline comments explaining *why* each block exists, not just what it does.

4.3 Prompts Used

Prompt 1 — Initial scraper generation (Ask mode):

Write a complete `scraper.py` using Python sync Playwright that scrapes `ekantipur.com`. Write code only, do not scrape. Zero hardcoded data values — everything comes from the page. [followed by complete selector specifications, output schema, and requirements as described in this document]

Prompt 2 — Comment enhancement (Agent mode):

The `scraper.py` is missing inline `#` comments inside function bodies. Add `#` comments before every `wait_for_selector` call, category extraction, scroll block, `try/except`, `.first` usage on cartoon, and caption parsing block. Each comment must explain WHY not just what. Do not change any code.

5 Code Explanation

5.1 Module Docstring and Imports

```
1  """
2  Scrape public listing metadata from ekantipur.com
   entertainment and cartoon
3  pages using synchronous Playwright: category, article titles,
   image URLs,
4  authors, and the featured cartoon caption. Writes structured
   JSON for
5  downstream use; no business data is embedded - all fields come
   from the live DOM.
6  """
7
8  import json
9  from urllib.parse import urlparse
10 from playwright.sync_api import sync_playwright
```

- `json` — built-in Python module for writing `output.json`
- `urlparse` — built-in URL parser used in the category fallback
- `sync_playwright` — the synchronous Playwright API; simpler than the async version for a single-purpose script

5.2 URL Constants

```
1 ENTERTAINMENT_URL = "https://ekantipur.com/entertainment"
2 CARTOON_URL       = "https://ekantipur.com/cartoon"
```

These are the only fixed strings in the file. They are task inputs (“where to scrape”), not scraped data values, so they do not violate the no-hardcoding requirement.

5.3 `_category_from_url` — URL Fallback

```
1 def _category_from_url(page_url: str) -> str:
2     """Derive a slug from the URL path when the header link is
3         missing."""
4     # Without this guard, malformed URLs or urlparse edge
5         cases would
6         abort the whole scrape.
7     try:
8         path = urlparse(page_url).path.strip("/")
9         segments = [s for s in path.split("/") if s]
10        return segments[0] if segments else "unknown"
11    except Exception:
12        return "unknown"
```

The underscore prefix (`_`) marks this as a private helper — not intended for external use. Given the URL `https://ekantipur.com/entertainment`:

1. `urlparse(...).path` extracts `/entertainment`
2. `.strip("/")` removes slashes → `entertainment`
3. `.split("/")` splits → `["entertainment"]`
4. `segments[0]` returns `"entertainment"`

This function is only called if `div.category-name p a` is not found in the page header.

5.4 `_scroll_in_six_steps` — Lazy Load Handler

```
1 def _scroll_in_six_steps(page) -> None:
2     """Lazy media often loads on scroll; incremental scroll
3         reduces missed assets."""
4     # Network or isolated-world failures would leave total
5         unset.
6     try:
7         total = page.evaluate("() =>
8             document.documentElement.scrollHeight")
9     except Exception:
10        total = 0
11    if not total:
12        return
13    # Images below the fold use lazy loading; stepping scroll
14        fires
15        intersection observers so src attributes can populate.
16    for step in range(6):
17        try:
18            y = int((total * step) / 5)
19            page.evaluate("(y) => window.scrollTo(0, y)", y)
20        except Exception:
```

```

17         pass
18     # Give the browser time to finish decoding lazy src values
19     # after the last scroll event.
20     try:
21         page.wait_for_timeout(2000)
22     except Exception:
23         pass

```

Why 6 steps? The scroll positions for a 3000px page would be: 0, 600, 1200, 1800, 2400, 3000px. This ensures every image on the page enters the viewport at some point, triggering the browser's `IntersectionObserver` callbacks that swap placeholder sources with real image URLs.

Why 2000 ms? Network latency plus image decoding time. Without this pause, `get_attribute("src")` may still return a placeholder immediately after the scroll.

5.5 `_parse_cartoon_caption` — Caption Parser

```

1 def _parse_cartoon_caption(raw: str) -> tuple[str, str | None]:
2     """Match site caption patterns: trailing dash vs ' - '
3     separator vs plain title."""
4     text = raw.strip()
5     # The markup uses one paragraph for both fields; we infer
6     # the author
7     # from the punctuation pattern the site uses in that
8     # string.
9     if text.endswith(" -"):
10         # Trailing dash with nothing after it means no author
11         # is listed.
12         return text[:-2].strip(), None
13     if " - " in text:
14         # Space-dash-space is the separator between title and
15         # cartoonist name.
16         first, second = text.split(" - ", 1)
17         author = second.strip() if second.strip() else None
18         return first.strip(), author
19     # No dash at all; the entire string is the title.
20     return text, None

```

The `endswith(" -")` check must come *before* the `" - "` in `text` check. If we checked the middle case first, the string `"Gajab chha ba! -"` would incorrectly trigger a split because `" -"` is contained within `" - "` (minus the trailing space).

5.6 `scrape_entertainment` — Main Extraction Function

```

1 def scrape_entertainment(page):
2     """Load entertainment listing and return up to five card
3     dicts."""
4     try:
5         page.goto(ENTERTAINMENT_URL,
6                   wait_until="domcontentloaded")

```

```

5     except Exception:
6         return [] # DNS/TLS/timeout failure: return empty,
                    # don't crash cartoon scrape
7
8     # Cards are injected after JS hydration; reading before
                    # this selector
9     # appears yields empty or stale locators.
10    try:
11        page.wait_for_selector("div.category-inner-wrapper",
                                timeout=60000)
12    except Exception:
13        return []
14
15    # Header category is stable before scroll; capture it
                    # first so lazy
16    # image churn cannot hide or reorder that node.
17    category = None
18    try:
19        el = page.locator("div.category-name p a").first
20        category = el.text_content().strip() if el else None
21    except Exception:
22        category = None
23    if not category:
24        category = _category_from_url(page.url)
25
26    _scroll_in_six_steps(page) # Trigger lazy image loading
27
28    articles = []
29    try:
30        wrappers = page.locator("div.category-inner-wrapper")
31        count = min(5, wrappers.count()) # Never extract more
                                        # than 5
32    except Exception:
33        count = 0
34
35    for i in range(count):
36        # ... per-card extraction (title, image_url, author)
37        ...
38    return articles

```

Key design decisions:

- `wait_until="domcontentloaded"` rather than `"networkidle"` — the entertainment page never fully reaches network idle due to continuous ad requests.
- Category is extracted *before* scrolling because the header node is always in the viewport and stable.
- `min(5, wrappers.count())` prevents index errors if the page has fewer than 5 cards.

5.7 scrape_cartoon — Cartoon Extraction

```
1 def scrape_cartoon(page):
2     """Load cartoon page and parse the first featured cartoon
3         block."""
4     try:
5         page.goto(CARTOON_URL, wait_until="domcontentloaded")
6     except Exception:
7         return {"title": None, "image_url": None, "author":
8             None}
9
10    # Cartoon assets render after layout; waiting avoids
11        reading empty
12        # wrappers on first paint.
13    try:
14        page.wait_for_selector("div.cartoon-wrapper",
15            timeout=60000)
16    except Exception:
17        return {"title": None, "image_url": None, "author":
18            None}
19
20    # The page lists many cartoons; the task wants only the
21        lead slot,
22        # so .first pins that single instance.
23    first = page.locator("div.cartoon-wrapper").first
24
25    img_el = first.locator("div.cartoon-image figure
26        img").first
27    image_url = img_el.get_attribute("src") if img_el else None
28
29    cap_el = first.locator("div.cartoon-description p").first
30    raw_caption = cap_el.text_content().strip() if cap_el else
31        ""
32
33    # Delegate caption string parsing to the dedicated helper.
34    title, author = _parse_cartoon_caption(raw_caption)
35    return {"title": title, "image_url": image_url, "author":
36        author}
```

5.8 main — Orchestration

```
1 def main() -> None:
2     """Drive browser sessions and persist JSON so results are
3         inspectable offline."""
4     payload = {
5         "entertainment_news": [],
6         "cartoon_of_the_day": {"title": None, "image_url":
7             None, "author": None},
8     }
9
10    with sync_playwright() as p:
11        browser = p.chromium.launch(headless=False) # Visible
12            browser per requirements
```

```
10     context = browser.new_context()    # Isolated browser
11         profile
12     page = context.new_page()
13     payload["entertainment_news"] =
14         scrape_entertainment(page)
15     payload["cartoon_of_the_day"] = scrape_cartoon(page)
16
17     context.close()
18     browser.close()
19
20     # ensure_ascii=False preserves Nepali Devanagari script
21     # as-is
22     # rather than converting to Unicode escape sequences.
23     with open("output.json", "w", encoding="utf-8") as f:
24         json.dump(payload, f, ensure_ascii=False, indent=2)
25
26 if __name__ == "__main__":
27     main()
```

Why `browser.new_context()` instead of directly calling `browser.new_page()`? A `BrowserContext` is an isolated session — separate cookies, cache, and local storage. This mirrors a clean browser profile, preventing any cached state from a previous run affecting the current one.

Why `if __name__ == "__main__":`? When this file is *run directly*, Python sets `__name__` to `"__main__"`, so `main()` is called. If this file is *imported* by another script (e.g., for unit testing individual functions), `__name__` equals the module name and `main()` is not triggered automatically.

6 Error Handling Strategy

Every element access in `scraper.py` is wrapped in a `try/except` block. The design principle is **partial data is better than no data**:

- If one article's image fails to load, the other four are still extracted.
- If the cartoon page times out, the entertainment data already collected is still written to `output.json`.
- If the author element is missing from a card, that field is set to `null` rather than raising an exception.
- If the browser binary is missing, `main()` returns gracefully rather than crashing with a traceback.

7 Output Format

7.1 Schema

```
{
  "entertainment_news": [
    {
      "title": "Article headline (Nepali text)",
      "image_url": "https://assets-cdn-api.ekantipur.com/thumb.php?src=...",
      "category": "Section name extracted from page header",
      "author": "Author name, or null if not present"
    }
  ],
  "cartoon_of_the_day": {
    "title": "Cartoon title parsed from caption paragraph",
    "image_url": "https://assets-cdn-api.ekantipur.com/thumb.php?src=...",
    "author": "Cartoonist name, or null if not listed"
  }
}
```

7.2 Sample Output

```
{
  "entertainment_news": [
    {
      "title": "Garibko Chameli: Balen sang it, it turned
        towards Balen",
      "image_url": "https://assets-cdn-api.ekantipur.com/thumb.php?src=...jpg",
      "category": "manoranjani",
      "author": "Reena Moktan"
    },
    ...
  ],
  "cartoon_of_the_day": {
    "title": "Gajab chha ba!",
    "image_url": "https://assets-cdn-api.ekantipur.com/thumb.php?src=...jpg",
    "author": null
  }
}
```

Encoding Note

The actual output contains Nepali Devanagari script. The JSON is written with `ensure_ascii=False` and `encoding="utf-8"`, which preserves characters such as *manoranjani* and *Gajab chha ba* in their native script rather than converting them to `\u` escape sequences.

8 Challenges and Solutions

Challenge	Root Cause	Solution
3 of 5 image URLs returning <code>null</code> on first run	Lazy loading — images only get a real <code>src</code> when scrolled into viewport	Scroll page in 6 incremental steps, wait 2000 ms
Cartoon title including trailing " -"	Some cartoons have no author but retain the dash separator in the text	Check <code>endswith(" -")</code> before the general split logic
Category field hardcoded	No per-card category label exists in the HTML	Extract category from the page header (<code>div.category-name p a</code>) dynamically
<code>uv</code> not recognised after restarting terminal	<code>PATH</code> set only for the current session	Use <code>[Environment]::SetEnvironmentVariable</code> to persist to user <code>PATH</code>
Nepali text appearing as <code>\u</code> escapes in JSON	Default <code>json.dump</code> uses <code>ensure_ascii=True</code>	Pass <code>ensure_ascii=False</code> and <code>encoding="utf-8"</code>

Table 3: Challenges encountered and their solutions

9 Submission Checklist

- ✓ Script runs without errors via `uv run python scraper.py`
- ✓ `output.json` contains valid JSON with proper UTF-8 encoding
- ✓ All 5 entertainment articles extracted with complete data
- ✓ Cartoon of the Day data complete
- ✓ Code has meaningful comments explaining *why*, not just *what*
- ✓ No hardcoded data values — all fields come from the live DOM
- ✓ `scraper.py` submitted
- ✓ `output.json` submitted
- ✓ `pyproject.toml` submitted
- ✓ `prompts.txt` submitted
- ✓ `recording.mp4` submitted

10 Running the Script

```
# Navigate to the project directory
cd C:\Users\<YourName>\Downloads\AudioBee\ekantipur-scraper

# Run the scraper
uv run python scraper.py
```

```
# Output is written to output.json in the same directory
```

A Chromium browser window will open visibly (`headless=False`), navigate to the entertainment section, scroll to trigger lazy loading, then navigate to the cartoon section, and finally close. The entire process takes approximately 10–15 seconds depending on network speed.